Compilation – Travaux pratiques

TP3 : Initiation à Flex

Générateur d'analyseurs lexicaux

ERRAMI YOUSSEF ILISI 2 – 2025/2026

1 Introduction

L'analyse lexicale est la première étape d'un compilateur : elle découpe le texte source en unités significatives (mots-clés, identificateurs, nombres, opérateurs...). Plutôt que de programmer cet automate à la main, on le décrit avec des *expressions régulières* et l'outil **Flex** génère le code C correspondant.

L'objectif de ce TP est de se familiariser avec Flex à travers trois séries d'exercices :

— reconnaître les éléments d'un petit langage de programmation (exercice 1.1) ;

— utiliser des variables C et des états dans les actions pour supprimer les commentaires (exercice 1.2) ;

— écrire des expressions régulières pour quatre langages sur l'alphabet $\{a, b\}$ ou les lettres (exercice 1.3).

Tous les exercices sont aussi regroupés dans un seul programme avec un menu (`tp3_menu.l`), ce qui permet de les tester facilement les uns après les autres.

2 Rappel : fonctionnement de Flex

Un fichier Flex comporte trois zones séparées par `%%` :

```
(definitions : options, macros, code C entre %{ et %})
%%
expression_reguliere { action en C }
%%
(code C : main, fonctions annexes)
```

Flex lit le texte et applique deux principes qu'il faut bien avoir en tête pour comprendre tous les résultats de ce TP :

1. **Plus longue correspondance** : parmi toutes les règles, Flex choisit celle qui reconnaît le plus de caractères.
2. **Priorité à l'ordre** : si deux règles reconnaissent la même longueur, c'est celle écrite en premier qui est choisie.

Un caractère qu'aucune règle ne reconnaît est simplement recopié sur la sortie (action par défaut `ECHO`).

Le texte reconnu est disponible dans la variable `yytext`. La chaîne de compilation est :

```
flex exercice.l (produit lex.yy.c)
gcc lex.yy.c -o exercice (compile l'analyseur)
./exercice < fichier.txt (l'exécute sur un fichier)
```

3 Exercice 1.1 : programme source d'un langage simple

Texte de test utilisé (fichier `0.dat`) :

```
for( i=0; i < 10; i++ ) {
  truc[i] = 0;
  machin = 12.3;
  bidule = .0;
}
```

3.1 a) Reconnaissance de symboles

La spécification demandée est une seule règle :

```
[a-zA-Z][a-zA-Z0-9]* { printf("<%s> ", yytext); }
```

Explication du résultat. La règle reconnaît tout mot qui commence par une lettre suivie de lettres ou de chiffres, et l'affiche entre chevrons. Tout le reste est recopié tel quel par Flex :

Entrée	Sortie
`for( i=0; i < 10; i++ ) {`	`<for>( <i>=0; <i> < 10; <i>++ ) {`
`truc[i] = 0;`	`<truc>[<i>] = 0;`
`machin = 12.3;`	`<machin> = 12.3;`
`bidule = .0;`	`<bidule> = .0;`

Les nombres `10`, `12.3` et `.0` ne sont pas reconnus car ils ne commencent pas par une lettre. On remarque aussi que `for` est traité comme un simple identificateur : rien ne le distingue encore des autres mots. C'est ce que corrige la question suivante.

3.2 b1) Mots-clés, opérateurs et symboles

On sépare maintenant trois catégories : mots-clés, symboles (identificateurs) et opérateurs.

```
for|while|if|else|do|int|float|char|return|... { printf("<MOT-CLE:%s> ", yytext); }
[a-zA-Z][a-zA-Z0-9]* { printf("<SYMBOLE:%s> ", yytext); }
"++"|"--"|"=="|"!="|"<="|">="|"&&"|"||"|... { printf("<OPERATEUR:%s> ", yytext); }
[-+*/%=<>!&|^~] et [(){}\[\];,.:] { printf("<OPERATEUR:%s> ", yytext); }
```

Le mot `for` est reconnu à la fois par la règle des mots-clés et par celle des symboles, avec la même longueur. C'est la priorité à l'ordre qui tranche : la règle des mots-clés doit donc être placée *avant*. De même, `++` est reconnu comme un seul opérateur et non comme deux `+`, parce que la règle à deux caractères donne la plus longue correspondance.

Sur `for( i=0; i < 10; i++ )`, on obtient : `<MOT-CLE:for> <OPERATEUR:(> <SYMBOLE:i> <OPERATEUR:=`
À ce stade les nombres sont encore recopiés sans traitement.

3.3 b2) Valeurs numériques

On ajoute deux règles : les entiers et les flottants (avec exposant éventuel).

```
EXP [eE][-+]?[0-9]+
[0-9]+ { printf("<ENTIER:%s> ", yytext); }
[0-9]+"."[0-9]*({EXP})?|"."[0-9]+({EXP})?|[0-9]+{EXP} { printf("<FLOTTANT:%s> ",
    yytext); }
```

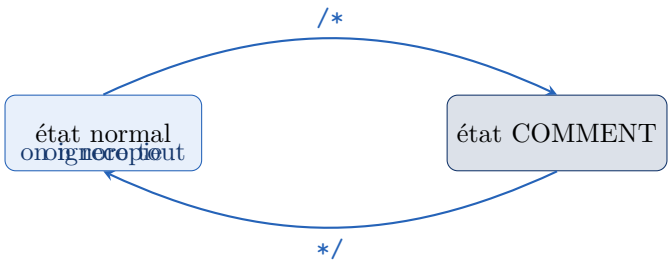
Un flottant peut s'écrire `12.3`, `12.`, `.0` ou `1e5`. Sur `12.3`, la règle des flottants (4 caractères) l'emporte sur celle des entiers (`12`, 2 caractères) : c'est encore la plus longue correspondance.

Entrée	Sortie
`machin = 12.3;`	`<SYMBOLE:machin> <OPERATEUR:=> <FLOTTANT:12.3> <OPERATEUR:;>`
`bidule = .0;`	`<SYMBOLE:bidule> <OPERATEUR:=> <FLOTTANT:.0> <OPERATEUR:;>`
`i++`	`<SYMBOLE:i> <OPERATEUR:++>`

4 Exercice 1.2 : suppression des commentaires C

Il s'agit de supprimer les commentaires `/* ... */`, qui peuvent s'étendre sur plusieurs lignes. Une seule expression régulière serait possible, mais lourde et peu lisible. On utilise plutôt une **condition de départ** de Flex, qui joue le rôle d'un état de l'analyseur.

```
%x COMMENT
%{ int nb_comm = 0, nb_lignes = 0; %}
%%
\"([^"\\\n]|\\.)*\" { ECHO; }
"/*" { BEGIN(COMMENT); nb_comm++; }
<COMMENT>"*/" { BEGIN(INITIAL); }
<COMMENT>\n { nb_lignes++; }
<COMMENT>. { /* on ignore */ }
<COMMENT><<EOF>> { fprintf(stderr, "commentaire non termine\n"); return 0; }
```



Logique.

— En état normal, le texte est recopié. La lecture de `/*` fait passer dans l'état `COMMENT`.

— Dans cet état, tous les caractères sont ignorés, y compris les retours à la ligne ; seul `*/` ramène à l'état normal. Cela gère naturellement les commentaires multi-lignes.

— **Variables dans les actions** : `nb_comm` et `nb_lignes` sont de simples variables C, modifiées dans les actions et affichées par `main` à la fin.

— La règle sur les chaînes `"..."` évite de supprimer à tort un `/*` situé dans un texte, par exemple `printf("a /* b */ c")`.

— Si le fichier se termine en plein commentaire, la règle `«EOF»` signale l'erreur.

Sur le fichier de test, 3 commentaires sont supprimés (dont un sur plusieurs lignes) et la chaîne qui contient un faux commentaire reste intacte.

5 Exercice 1.3 : expressions régulières

Quatre analyseurs reconnaissent chacun un langage. Les hypothèses du sujet sont respectées : un seul mot par ligne, les délimiteurs (espace, tabulation, fin de ligne) sont simplement recopiés, et une règle « attrape-tout » signale chaque caractère d'un mot non reconnu.

```
\n { ECHO; }
[ \t] { ECHO; }
^ ER $ { printf("%s -> ACCEPTE", yytext); }
. { printf("<?%c>", yytext[0]); }
```

Sans ancrage, Flex pourrait reconnaître seulement un *morceau* du mot. Exemple pour le langage c) : dans `abba`, le morceau final `ba` est valide et serait accepté à tort. En imposant le début (`^`) et la fin de ligne (`$`), on vérifie le mot entier.

5.1 a) Les cinq voyelles dans l'ordre

```
^{L}*[aA]{L}*[eE]{L}*[iI]{L}*[oO]{L}*[uU]{L}*$ avec L = [a-zA-Z]
```

Les lettres sont libres entre les voyelles, qui doivent seulement apparaître dans l'ordre a, e, i, o, u.

5.2 b) Mots commençant ou finissant par `a`

```
^(a[ab]*|[ab]*a)$
```

Deux cas reliés par `|` : un `a` suivi de n'importe quoi, ou n'importe quoi suivi d'un `a`.

5.3 c) Pas deux caractères identiques consécutifs

```
^((ab)+a?|(ba)+b?|a|b)$
```

Si deux lettres voisines sont toujours différentes, le mot *alterne* : `abab...` ou `baba...`. On écrit ces deux familles avec une lettre finale facultative, plus les mots d'une lettre.

5.4 d) Mots sans la sous-chaîne `aba`

```
^(b+)?(a+bb+)*(a+b?)?$
```

C'est l'expression la moins évidente. Interdire `aba` revient à interdire un **seul** `b` entre deux `a`. Donc tout bloc de `b` placé entre deux blocs de `a` doit en contenir au moins deux. Le mot se découpe ainsi :

1. des `b` au début, éventuellement : `(b+)?` ;
2. des groupes « un bloc de `a`, puis au moins deux `b` » : `(a+bb+)*` ;
3. un dernier bloc de `a`, éventuellement suivi d'un seul `b` (permis car rien ne suit) : `(a+b?)?`.

5.5 Résultats des tests

Langage	Mots acceptés	Mots rejetés
a) voyelles	`abstemious, facetious, aeiou, AEIOU`	`sequoia, banane, education`
b) a au bord	`a, ab, ba, abb, aba, abbb, bba`	`b, bab, bbb`
c) alternés	`a, b, ab, abab, baba, babab`	`abba, bb, aab`
d) sans aba	`a, b, ab, abba, bbaa, abb, bbabb, aabba`	`aba, abab, baba, bab, abaa`

Pour un mot rejeté comme `abba` dans le langage c), la sortie est `<?a><?b><?b><?a>` : la règle attrape-tout signale chaque caractère.

6 Programme avec menu

Pour faciliter le test final, `tp3_menu.l` regroupe tous les exercices dans un seul analyseur. Chaque exercice est une condition de départ (`E11A`, `E11B1`, `E11B2`, `E12`, `E13A` à `E13D`) et ses règles sont préfixées par cette condition, par exemple `<E13B>^(a[ab]*|[ab]*a)$`.

Dans `main`, le programme :

1. affiche le menu et lit le choix de l'utilisateur ;
2. charge le texte, soit depuis un fichier, soit depuis une saisie au clavier ;
3. confie ce texte à l'analyseur avec `yy_scan_string` ;
4. active l'exercice choisi avec `BEGIN(condition)`, puis appelle `yylex()`.

Le texte est nettoyé avant l'analyse (suppression des `\r` des fichiers Windows, ajout d'un retour à la ligne final) afin que les règles ancrées en fin de ligne fonctionnent toujours.

7 Difficultés rencontrées

— **Mots acceptés à tort** (exercice 1.3) : sans ancrage, un morceau de mot était reconnu. L'ancrage par `^` et `$` règle le problème.

— **Fichiers Windows** : les retours à la ligne `\r\n` empêchaient la reconnaissance de fin de ligne ; on les supprime avant l'analyse.

— **Commentaires dans les chaînes** (exercice 1.2) : une règle dédiée aux chaînes empêche de les supprimer par erreur.

— **Ordre des règles** (exercice 1.1) : mots-clés avant identificateurs, sinon `for` serait un simple symbole.